

Lecture 10: Introduction to graph algorithms

*Instructor: Sourav Chakraborty**Scribe: Rupayan Mandal, Shrabanti Das*

1 Some basic terminologies

1.1 Basic Structure

Graph: A graph is an ordered pair $G = (V, E)$ where:

- V is a nonempty finite set of vertices.
- $E \subseteq \{\{u, v\} \mid u, v \in V\}$ is a set of edges (for undirected graphs).

For directed graphs, $E \subseteq \{(u, v) \mid u, v \in V\}$.

Vertex (Node): An element of the set V .

Edge: An element of the set E connecting two vertices.

1.2 Types of Graphs

- **Undirected Graph:** A graph in which edges are unordered pairs $\{u, v\}$.
- **Directed Graph (Digraph):** A graph in which edges are ordered pairs (u, v) .
- **Simple Graph:** An undirected graph with no loops and no multiple edges.
- **Multigraph:** A graph that allows multiple edges between the same pair of vertices.
- **Pseudograph:** A graph that allows both loops and multiple edges.
- **Weighted Graph:** A graph in which each edge is assigned a numerical value (weight).

1.3 Degree and Adjacency

- **Degree of a Vertex** ($\deg(v)$): The number of edges incident with vertex v . (A loop counts twice in an undirected graph.)
- **In-degree** ($\deg^-(v)$): Number of edges entering vertex v in a directed graph.
- **Out-degree** ($\deg^+(v)$): Number of edges leaving vertex v in a directed graph.
- **Adjacent Vertices**: Vertices u and v are adjacent if $\{u, v\} \in E$.
- **Incident**: An edge is incident to a vertex if the vertex is one of its endpoints.
- **Isolated Vertex**: A vertex of degree 0.
- **Pendant (Leaf) Vertex**: A vertex of degree 1.

1.4 Walks and Paths

- **Walk**: A finite sequence of vertices $v_0, v_1, \dots, v_k$ such that each consecutive pair is connected by an edge.
- **Trail**: A walk with no repeated edges.
- **Path**: A walk with no repeated vertices.
- **Simple Path**: A path in which all vertices are distinct.
- **Closed Walk**: A walk where $v_0 = v_k$.
- **Cycle (Circuit)**: A closed path with no repeated vertices except the first and last.

1.5 Special Graphs

- **Complete Graph (K_n):** A simple graph in which every pair of distinct vertices is adjacent.
- **Bipartite Graph:** A graph whose vertex set can be partitioned into two disjoint sets V_1 and V_2 such that every edge connects a vertex in V_1 to one in V_2 .
- **Tree:** A connected graph with no cycles.
- **Cycle Graph (C_n):** A graph consisting of a single cycle of length n .

1.6 Clique and Spanning Tree

Let $G = (V, E)$ be a simple undirected graph.

1.6.1 Clique

A subset $C \subseteq V$ is called a **clique** if

$$\forall u, v \in C, u \neq v \Rightarrow \{u, v\} \in E.$$

Equivalently, C is a clique if the induced subgraph $G[C]$ is complete.

The **clique number** of G is defined as

$$\omega(G) = \max\{ |C| : C \subseteq V \text{ and } C \text{ is a clique} \}.$$

1.6.2 Spanning Tree

A subgraph $T = (V, E_T)$ is called a **spanning tree** of G if:

1. $E_T \subseteq E$,
2. T is connected,
3. T is acyclic.

Equivalently, T is a spanning tree of G if

$$|E_T| = |V| - 1 \quad \text{and} \quad T \text{ is connected.}$$

Equivalent Characterization:

A subgraph T of G is a spanning tree if and only if it is *minimal connected*, or *maximal acyclic*.

1.7 Connectivity

- **Connected Graph:** A graph in which there exists a path between every pair of vertices.
- **Disconnected Graph:** A graph that is not connected.
- **Component:** A maximal connected subgraph.

1.8 Matrix Representation

- **Adjacency Matrix:** An $n \times n$ matrix $A = [a_{ij}]$ where

$$a_{ij} = \begin{cases} 1 & \text{if there is an edge between } v_i \text{ and } v_j, \\ 0 & \text{otherwise.} \end{cases}$$

- **Incidence Matrix:** A matrix representing the relationship between vertices and edges.

Dijkstra's Algorithm And Prim's Algorithm

1 . Problem

Given a weighted graph $G = (V, E)$ with non-negative edge weights, find the shortest path from source vertex s to all other vertices.

Algorithm

Algorithm 1 Dijkstra(G, s)

```
For each vertex  $v \in V$ , set  $dist[v] = \infty$ 
 $dist[s] = 0$ 
Mark all vertices as unvisited
Insert  $(0, s)$  into priority queue  $Q$ 
while  $Q$  is not empty do
    Extract vertex  $u$  with minimum distance
    if  $u$  is already visited then
        Continue
    end if
    Mark  $u$  as visited
    for each neighbor  $v$  of  $u$  do
        if  $dist[v] > dist[u] + w(u, v)$  then
             $dist[v] = dist[u] + w(u, v)$ 
            Insert  $(dist[v], v)$  into  $Q$ 
        end if
    end for
end while
return  $dist$ 
```

Time Complexity

- Using Adjacency Matrix: $O(V^2)$
- Using Adjacency List + Min Heap: $O((V + E) \log V)$

Space Complexity

- Distance array: $O(V)$
- Priority queue: $O(V)$
- Graph storage:
 - Matrix: $O(V^2)$
 - List: $O(V + E)$

2 . Problem

Given a connected, undirected weighted graph $G = (V, E)$, find a Minimum Spanning Tree (MST).

Prim's Algorithm

Working Principle

Prim's algorithm is a **Greedy Algorithm**. At each step, it selects the minimum weight edge that connects a vertex inside the MST to a vertex outside the MST.

Time Complexity

Let $|V|$ be number of vertices and $|E|$ number of edges.

- Using Adjacency Matrix:

$$O(V^2)$$

- Using Adjacency List + Min Heap:

$$O((V + E) \log V)$$

Space Complexity

- Key array: $O(V)$
- Parent array: $O(V)$
- Priority Queue: $O(V)$
- Graph storage:
 - Matrix: $O(V^2)$
 - List: $O(V + E)$